

Phase Execution Document

E-Commerce Laravel Architecture Implementation

LUWI Mafuleti

May 18, 2026

Contents

1	Project Initialization Commands	6
1.1	Laravel Installation	6
1.2	Development Branch Setup	6
2	System Architecture (MCD)	6
2.1	Conceptual Diagram	6
3	Database Schema (Laravel Migration Implementation)	6
3.1	CORE TABLES	6
3.1.1	1. USERS	6
3.1.2	2. CATEGORIES	7
3.1.3	3. PRODUCTS	7
3.1.4	4. CART	7
3.1.5	5. CART ITEMS	7
3.1.6	6. ORDERS	8
3.1.7	7. ORDER ITEMS	8
3.2	ADVANCED (FUTURE LAYER)	8
3.2.1	8. PRODUCT IMAGES	8
3.2.2	9. PRODUCT VARIANTS	9
3.2.3	10. REVIEWS	9
3.2.4	11. PAYMENTS	9
3.2.5	12. ADDRESSES	10
3.2.6	13. VENDORS	10
3.2.7	14. VENDOR PRODUCTS	10
4	Phase Execution	10
4.1	Step 2: Implement Models and Relationships (Eloquent ORM)	10
4.1.1	1. Model Creation Commands	11
4.1.2	2. Core Eloquent Relationships	11

5	Phase Execution	15
5.1	Step 3: Implement Controllers (MVC Architecture)	15
5.1.1	1. Controller Generation Commands	15
5.1.2	2. MVC Controller Design Principles	15
5.1.3	3. Example: ProductController (Web MVC)	15
5.1.4	4. Example: CartController (Session-Based Logic)	17
5.1.5	5. Example: OrderController (Checkout Flow)	18
5.1.6	6. CategoryController	19
5.1.7	7. ReviewController	20
5.1.8	8. AddressController	21
5.1.9	9. VendorController	21
5.1.10	10. PaymentController	22
5.2	Refined Routing & Controller Hardening	23
5.2.1	Consolidated Routing Architecture	23
5.2.2	Order Lifecycle and Stock Restoration	23
5.2.3	Discovery Engine: Search and Filtering	23
5.2.4	6. Production MVC Best Practices	23
5.2.5	7. Next Phase Notes	23
6	Next Phase: System Enhancement and Hardening	24
6.1	5.1.12 Production Notes (ReviewController)	24
7	Phase Execution: Frontend and Application Layer	24
7.1	1. Blade UI Templates (View Layer Implementation)	24
7.1.1	Layout Structure	24
7.1.2	Main Layout Example	24
7.2	2. Middleware Implementation (Authentication and Roles)	25
7.2.1	Middleware Creation	25
7.2.2	Middleware Logic	25
7.3	Security Layer: Advanced Form Requests	25
7.3.1	Controller Usage	25
7.4	4. File Upload Handling (Product Images)	25
7.4.1	Upload Implementation	26
7.4.2	Display in Blade	26
7.5	5. Production Considerations	26
7.6	Transition to Full Application Layer	26
8	Blade UI System Implementation	26
8.1	1. View Architecture Overview	26
8.2	2. Main Layout (Master Template)	27
8.3	3. Navigation Bar (Reusable Partial)	27
8.4	4. Product Listing View (Grid System)	27
8.5	5. Product Card Component (Reusable UI)	28
8.6	6. Product Detail Page	28
8.7	7. Dashboard Structure	28
8.8	8. Blade Concepts Used	29
8.9	9. Production UI Best Practices	29

9	Routing Architecture (web.php)	29
9.1	1. Overview	29
9.2	2. Basic Route Structure	29
9.3	3. Public Routes (Frontend Pages)	30
9.4	4. Authentication-Protected Routes	30
9.5	5. Admin Routes (Role-Based Access)	30
9.6	6. Resource Controllers	30
9.7	7. Named Routes	31
9.8	8. Middleware Usage	31
9.9	9. Production Routing Best Practices	31
10	Authorization Policies (Access Control Layer)	31
10.1	1. Overview	31
10.2	2. Policy Creation	31
10.3	3. ProductPolicy Implementation	32
10.4	4. ReviewPolicy Implementation	32
10.5	5. Policy Registration	32
10.6	6. Using Policies in Controllers	32
10.7	7. Blade Authorization	33
10.8	8. Security Benefits	33
10.9	9. Production Best Practices	33
11	System Architecture Summary	33
11.1	1. Overview	33
11.2	2. Architectural Layers	34
11.2.1	2.1 Presentation Layer (View)	34
11.2.2	2.2 Application Layer (Controllers)	34
11.2.3	2.3 Domain Layer (Models)	34
11.2.4	2.4 Data Layer (Database)	34
11.3	3. Cross-Cutting Concerns	34
11.3.1	3.1 Routing	34
11.3.2	3.2 Middleware	34
11.3.3	3.3 Validation Layer	34
11.3.4	3.4 Authorization	35
11.4	4. Data Flow Execution	35
11.5	5. Security Architecture	35
11.6	6. Scalability and Maintainability	35
11.7	7. Final System Perspective	35
12	Database Seeding and Test Data Initialization	36
12.1	1. Seeding Strategy (Modular Approach)	36
12.2	2. DatabaseSeeder (Main Entry Point)	36
12.3	3. UserSeeder (Role-Based Data)	36
12.4	4. CategorySeeder	37
12.5	5. ProductSeeder	37
12.6	6. ReviewSeeder (Policy Testing Data)	38
12.7	7. OrderSeeder	38
12.8	8. Running Seeders	39

13 E-Commerce Platform: Final Configuration and Bug Fixes	39
13.1 1. Problem Identification	39
13.2 2. Solutions and Implementation	39
13.2.1 2.1 Data Retrieval in ViewController	39
13.2.2 2.2 Implementing the Shop View	40
13.2.3 2.3 Fixing ProductController Logic	40
13.2.4 2.4 Administrative View Structure	40
13.3 3. Critical Observations	41
13.3.1 3.1 Middleware Configuration	41
13.3.2 3.2 Model Relationship Correction	41
13.4 4. Outcome	41
14 Frontend Enhancement and Feature Expansion Plan	41
14.1 Option A — User Interface (UI) Improvements	41
14.2 Option B — Functional Feature Enhancements	42
14.3 Option C — Advanced Production-Level Features	42
14.4 Authentication Module Adaptation (Legacy Integration)	42
14.4.1 Login Interface	42
14.4.2 Sign Up Interface	43
14.5 Implementation Strategy	44
15 Authentication System Completion and Validation	44
15.1 1. Problem Identification	44
15.2 2. Required Fixes and Implementation	44
15.2.1 2.1 Missing Authentication Controller	44
15.2.2 2.2 Password Hashing Verification	45
15.2.3 2.3 Missing POST Routes	45
15.2.4 2.4 Role Management Strategy	45
15.2.5 2.5 Layout Adaptation for Authentication State	45
15.2.6 2.6 Session and Validation Feedback	46
15.3 3. Outcome	46
16 Implementation Progress: Authentication and User Lifecycle Management	46
16.1 Completed Milestones	46
16.1.1 1. Authentication Controller (AuthController)	46
16.1.2 2. Secure Data Handling and Model Configuration	47
16.1.3 3. Routing and Middleware Architecture	47
16.1.4 4. Dynamic Layout and Session Awareness	47
16.1.5 5. Backend Integration for Product Display	47
16.1.6 6. Database Seeding and Testing Infrastructure	47
16.1.7 7. Error Handling and Validation Layer	48
16.2 System Status	48
16.3 Remaining Implementation Scope	48
17 Cart and Checkout System Implementation	48
17.1 1. Overview	48
17.2 2. Cart Management Logic	48
17.2.1 Core Behaviors	48

17.2.2	Cart Initialization (Critical)	49
17.3	3. Checkout and Order Creation	49
17.3.1	Checkout Flow	49
17.3.2	Order Creation Logic	49
17.4	4. Data Integrity and Validation	49
17.4.1	Stock Protection	49
17.4.2	Empty Cart Prevention	49
17.5	5. System Integration	50
17.6	6. Outcome	50
18	Shopping Cart and Order Processing System	50
18.1	Overview	50
18.2	Cart Management Implementation	50
18.2.1	Core Features	50
18.2.2	Cart Initialization Logic	50
18.2.3	Stock Protection	51
18.3	Order Processing (Checkout Flow)	51
18.3.1	Checkout Workflow	51
18.3.2	Order Creation	51
18.3.3	Data Migration (Cart → Order)	51
18.3.4	Stock Update	51
18.3.5	Cart Cleanup	51
18.4	Data Integrity and Transaction Safety	52
18.5	User Experience Considerations	52
18.6	System Readiness	52
19	Controller Methods and Laravel Terminology	52
19.1	Core Helper Functions	52
19.2	Eloquent ORM Methods	53
19.3	Request Handling	53
19.4	Authentication	53
19.5	Session Flash Messages	53
19.6	Routing Integration	53
20	Professional UI/UX Implementation Summary	54
20.1	Core Design System	54
20.2	Robust Image & Asset Handling	54
20.3	Enhanced Component Lifecycle	54
20.4	Luxury-Tier UI Finalization	54
20.4.1	Public Storytelling and Support	55
20.4.2	Executive Administrative Dashboard	55
20.4.3	UI Stability and Grid Logic	55
21	Conclusion	55

Project Initialization Commands

Laravel Installation

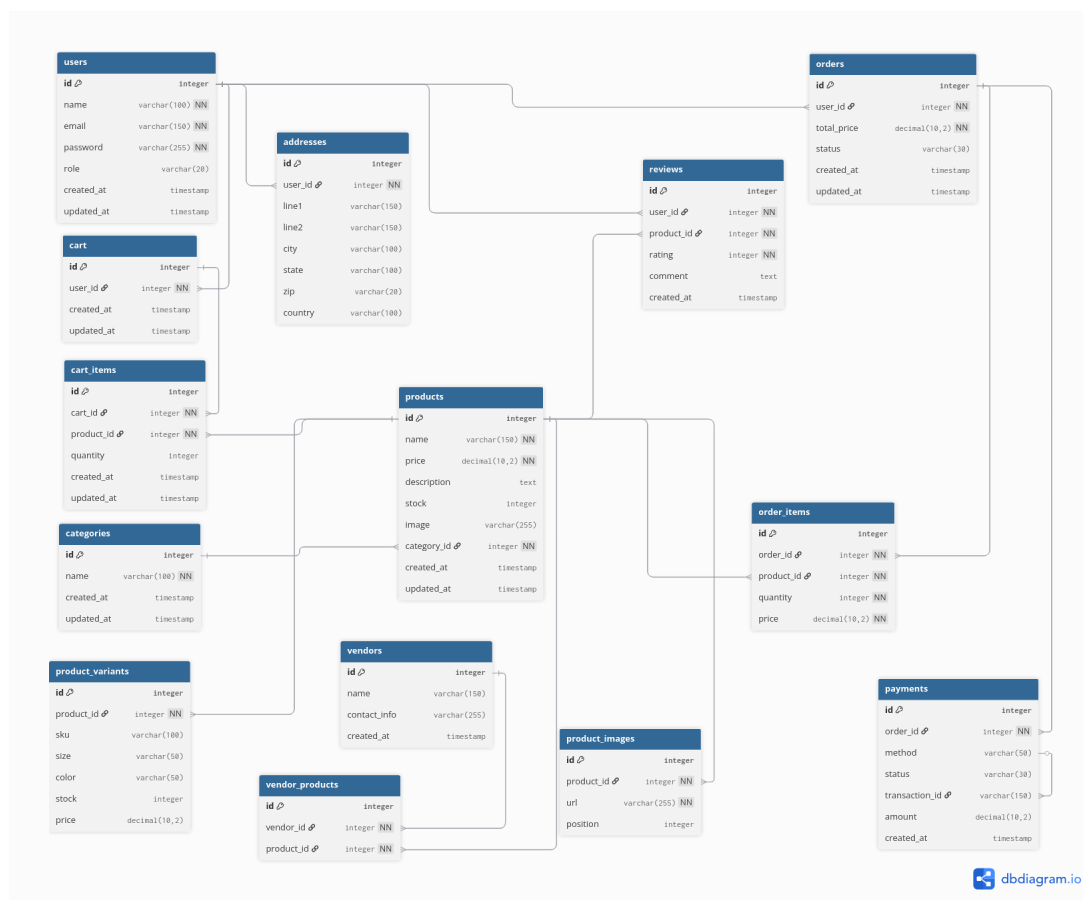
```
composer create-project laravel/laravel e-commerce-platform
```

Development Branch Setup

```
composer create-project laravel/laravel "E-Commerce Platform Architecture" dev-develop
```

System Architecture (MCD)

Conceptual Diagram



Database Schema (Laravel Migration Implementation)

This section reflects the actual Laravel migration structure implemented in the project using artisan commands and Schema Builder.

CORE TABLES

3.1.1 1. USERS

```
php artisan make:migration create_users_table
```

```
Schema::create('users', function (Blueprint $table) {
    $table->id();
    $table->string('name', 100);
    $table->string('email', 150)->unique();
    $table->string('password');
    $table->string('role')->default('user');
    $table->timestamps();
});
```

3.1.2 2. CATEGORIES

```
php artisan make:migration create_categories_table
```

```
Schema::create('categories', function (Blueprint $table) {
    $table->id();
    $table->string('name', 100);
    $table->timestamps();
});
```

3.1.3 3. PRODUCTS

```
php artisan make:migration create_products_table
```

```
Schema::create('products', function (Blueprint $table) {
    $table->id();
    $table->string('name', 150);
    $table->decimal('price', 10, 2);
    $table->text('description')->nullable();
    $table->integer('stock')->default(0);
    $table->string('image')->nullable();

    $table->foreignId('category_id')->constrained('categories');

    $table->timestamps();
});
```

3.1.4 4. CART

```
php artisan make:migration create_cart_table
```

```
Schema::create('cart', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained('users');
    $table->timestamps();
});
```

3.1.5 5. CART ITEMS

```
php artisan make:migration create_cart_items_table
```

```
Schema::create('cart_items', function (Blueprint $table) {
    $table->id();

    $table->foreignId('cart_id')->constrained('cart');
    $table->foreignId('product_id')->constrained('products');

    $table->integer('quantity')->default(1);
    $table->timestamps();
});
```

3.1.6 6. ORDERS

```
php artisan make:migration create_orders_table
```

```
Schema::create('orders', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained('users');

    $table->decimal('total_price', 10, 2);
    $table->string('status')->default('pending');

    $table->timestamps();
});
```

3.1.7 7. ORDER ITEMS

```
php artisan make:migration create_order_items_table
```

```
Schema::create('order_items', function (Blueprint $table) {
    $table->id();

    $table->foreignId('order_id')->constrained('orders');
    $table->foreignId('product_id')->constrained('products');

    $table->integer('quantity');
    $table->decimal('price', 10, 2);
});
```

ADVANCED (FUTURE LAYER)

3.2.1 8. PRODUCT IMAGES

```
php artisan make:migration create_product_images_table
```

```
Schema::create('product_images', function (Blueprint $table) {
    $table->id();
    $table->foreignId('product_id')->constrained('products');

    $table->string('url');
    $table->integer('position')->default(0);
});
```


3.2.2 9. PRODUCT VARIANTS

```
php artisan make:migration create_product_variants_table
```

```
Schema::create('product_variants', function (Blueprint $table) {  
    $table->id();  
    $table->foreignId('product_id')->constrained('products');  
  
    $table->string('sku')->nullable();  
    $table->string('size')->nullable();  
    $table->string('color')->nullable();  
  
    $table->integer('stock')->default(0);  
    $table->decimal('price', 10, 2)->nullable();  
});
```

3.2.3 10. REVIEWS

```
php artisan make:migration create_reviews_table
```

```
Schema::create('reviews', function (Blueprint $table) {  
    $table->id();  
  
    $table->foreignId('user_id')->constrained('users');  
    $table->foreignId('product_id')->constrained('products');  
  
    $table->integer('rating');  
    $table->text('comment')->nullable();  
  
    $table->timestamps();  
});
```

3.2.4 11. PAYMENTS

```
php artisan make:migration create_payments_table
```

```
Schema::create('payments', function (Blueprint $table) {  
    $table->id();  
  
    $table->foreignId('order_id')->constrained('orders');  
  
    $table->string('method')->nullable();  
    $table->string('status')->nullable();  
    $table->string('transaction_id')->nullable();  
    $table->decimal('amount', 10, 2)->nullable();  
  
    $table->timestamps();  
});
```

3.2.5 12. ADDRESSES

```
php artisan make:migration create_addresses_table
```

```
Schema::create('addresses', function (Blueprint $table) {
    $table->id();

    $table->foreignId('user_id')->constrained('users');

    $table->string('line1')->nullable();
    $table->string('line2')->nullable();
    $table->string('city')->nullable();
    $table->string('state')->nullable();
    $table->string('zip')->nullable();
    $table->string('country')->nullable();
});
```

3.2.6 13. VENDORS

```
php artisan make:migration create_vendors_table
```

```
Schema::create('vendors', function (Blueprint $table) {
    $table->id();
    $table->string('name')->nullable();
    $table->string('contact_info')->nullable();
    $table->timestamps();
});
```

3.2.7 14. VENDOR PRODUCTS

```
php artisan make:migration create_vendor_products_table
```

```
Schema::create('vendor_products', function (Blueprint $table) {
    $table->id();

    $table->foreignId('vendor_id')->constrained('vendors');
    $table->foreignId('product_id')->constrained('products');
});
```

Phase Execution

Step 2: Implement Models and Relationships (Eloquent ORM)

After completing database migrations, the next step is to define Laravel Eloquent models and establish relationships between entities. This allows the application to interact with the database using an Object Relational Mapping (ORM) approach instead of raw SQL queries.

4.1.1 1. Model Creation Commands

Each database table has a corresponding Eloquent model:

```
php artisan make:model User
php artisan make:model Category
php artisan make:model Product
php artisan make:model Cart
php artisan make:model CartItem
php artisan make:model Order
php artisan make:model OrderItem
php artisan make:model ProductImage
php artisan make:model ProductVariant
php artisan make:model Review
php artisan make:model Payment
php artisan make:model Address
php artisan make:model Vendor
php artisan make:model VendorProduct
```

4.1.2 2. Core Eloquent Relationships

User Model

```
class User extends Authenticatable
{
    public function orders()
    {
        return $this->hasMany(Order::class);
    }

    public function cart()
    {
        return $this->hasOne(Cart::class);
    }

    public function addresses()
    {
        return $this->hasMany(Address::class);
    }

    public function reviews()
    {
        return $this->hasMany(Review::class);
    }
}
```

Category Model

```
class Category extends Model
{
```

```

    public function products()
    {
        return $this->hasMany(Product::class);
    }
}

```

Product Model

```

class Product extends Model
{
    public function category()
    {
        return $this->belongsTo(Category::class);
    }

    public function cartItems()
    {
        return $this->hasMany(CartItem::class);
    }

    public function orderItems()
    {
        return $this->hasMany(OrderItem::class);
    }

    public function images()
    {
        return $this->hasMany(ProductImage::class);
    }

    public function variants()
    {
        return $this->hasMany(ProductVariant::class);
    }

    public function reviews()
    {
        return $this->hasMany(Review::class);
    }

    public function vendors()
    {
        return $this->belongsToMany(Vendor::class, 'vendor_products');
    }
}

```

Cart Model

```

class Cart extends Model

```

```

{
    public function user()
    {
        return $this->belongsTo(User::class);
    }

    public function items()
    {
        return $this->hasMany(CartItem::class);
    }
}

```

CartItem Model

```

class CartItem extends Model
{
    public function cart()
    {
        return $this->belongsTo(Cart::class);
    }

    public function product()
    {
        return $this->belongsTo(Product::class);
    }
}

```

Order Model

```

class Order extends Model
{
    public function user()
    {
        return $this->belongsTo(User::class);
    }

    public function items()
    {
        return $this->hasMany(OrderItem::class);
    }

    public function payment()
    {
        return $this->hasOne(Payment::class);
    }
}

```

OrderItem Model

```

class OrderItem extends Model
{
    public function order()
    {
        return $this->belongsTo(Order::class);
    }

    public function product()
    {
        return $this->belongsTo(Product::class);
    }
}

```

Payment Model

```

class Payment extends Model
{
    public function order()
    {
        return $this->belongsTo(Order::class);
    }
}

```

Review Model

```

class Review extends Model
{
    public function user()
    {
        return $this->belongsTo(User::class);
    }

    public function product()
    {
        return $this->belongsTo(Product::class);
    }
}

```

Vendor Model

```

class Vendor extends Model
{
    public function products()
    {
        return $this->belongsToMany(Product::class, 'vendor_products');
    }
}

```

Phase Execution

Step 3: Implement Controllers (MVC Architecture)

In this phase, controllers are implemented using Laravel's **MVC architecture**, focusing on server-rendered views (Blade templates) and session-based interactions.

These controllers handle HTTP requests, coordinate business logic, and return **views instead of JSON responses**.

5.1.1 1. Controller Generation Commands

All controllers are generated using Artisan:

```
php artisan make:controller UserController
php artisan make:controller CategoryController
php artisan make:controller ProductController
php artisan make:controller CartController
php artisan make:controller OrderController
php artisan make:controller PaymentController
php artisan make:controller ReviewController
php artisan make:controller AddressController
php artisan make:controller VendorController
```

5.1.2 2. MVC Controller Design Principles

All controllers follow these production MVC principles:

- Controllers return **Blade views** instead of JSON responses
- Business logic remains minimal inside controllers
- Data is passed using `compact()` or `with()`
- Session-based authentication (Auth facade)
- Validation handled using **Request validation**

5.1.3 3. Example: ProductController (Web MVC)

```
class ProductController extends Controller
{
    public function index()
    {
        $products = Product::with('category')->paginate(12);

        return view('products.index', compact('products'));
    }

    public function create()
    {
        $categories = Category::all();

        return view('products.create', compact('categories'));
    }
}
```

```

public function store(Request $request)
{
    $validated = $request->validate([
        'name' => 'required|string|max:150',
        'price' => 'required|numeric',
        'category_id' => 'required|exists:categories,id',
        'stock' => 'nullable|integer'
    ]);

    Product::create($validated);

    return redirect()
        ->route('products.index')
        ->with('success', 'Product created successfully');
}

public function show($id)
{
    $product = Product::with(['category', 'images', 'reviews.user'])
        ->findOrFail($id);

    return view('products.show', compact('product'));
}

public function edit($id)
{
    $product = Product::findOrFail($id);
    $categories = Category::all();

    return view('products.edit', compact('product', 'categories'));
}

public function update(Request $request, $id)
{
    $product = Product::findOrFail($id);

    $product->update($request->all());

    return redirect()
        ->route('products.index')
        ->with('success', 'Product updated successfully');
}

public function destroy($id)
{
    Product::destroy($id);
}

```



```

        return redirect()
            ->route('products.index')
            ->with('success', 'Product deleted successfully');
    }
}

```

5.1.4 4. Example: CartController (Session-Based Logic)

```

class CartController extends Controller
{
    public function index()
    {
        $cart = Cart::with('items.product')
            ->where('user_id', auth()->id())
            ->first();

        return view('cart.index', compact('cart'));
    }

    public function add(Request $request)
    {
        $request->validate([
            'product_id' => 'required|exists:products,id',
            'quantity' => 'required|integer|min:1'
        ]);

        $cart = Cart::firstOrCreate([
            'user_id' => auth()->id()
        ]);

        CartItem::updateOrCreate(
            [
                'cart_id' => $cart->id,
                'product_id' => $request->product_id
            ],
            [
                'quantity' => $request->quantity
            ]
        );

        return redirect()
            ->back()
            ->with('success', 'Product added to cart');
    }

    public function remove($id)
    {
        CartItem::destroy($id);
    }
}

```

```

        return redirect()
            ->back()
            ->with('success', 'Item removed from cart');
    }
}

```

5.1.5 5. Example: OrderController (Checkout Flow)

```

class OrderController extends Controller
{
    public function index()
    {
        $orders = Order::where('user_id', auth()->id())
                        ->latest()
                        ->get();

        return view('orders.index', compact('orders'));
    }

    public function store()
    {
        $cart = Cart::with('items.product')
                    ->where('user_id', auth()->id())
                    ->first();

        if (!$cart || $cart->items->isEmpty()) {
            return redirect()
                ->back()
                ->with('error', 'Cart is empty');
        }

        $total = 0;

        foreach ($cart->items as $item) {
            $total += $item->product->price * $item->quantity;
        }

        $order = Order::create([
            'user_id' => auth()->id(),
            'total_price' => $total,
            'status' => 'pending'
        ]);

        foreach ($cart->items as $item) {
            OrderItem::create([
                'order_id' => $order->id,
                'product_id' => $item->product_id,
            ]);
        }
    }
}

```

```

        'quantity' => $item->quantity,
        'price' => $item->product->price
    ]));
}

$cart->items()->delete();

return redirect()
    ->route('orders.index')
    ->with('success', 'Order placed successfully');
}
}

```

5.1.6 6. CategoryController

```

class CategoryController extends Controller
{
    public function index()
    {
        $categories = Category::latest()->get();
        return view('categories.index', compact('categories'));
    }

    public function create()
    {
        return view('categories.create');
    }

    public function store(Request $request)
    {
        $request->validate([
            'name' => 'required|string|max:100'
        ]);

        Category::create($request->all());

        return redirect()->route('categories.index')
            ->with('success', 'Category created successfully');
    }

    public function edit($id)
    {
        $category = Category::findOrFail($id);
        return view('categories.edit', compact('category'));
    }

    public function update(Request $request, $id)
    {

```

```

    $category = Category::findOrFail($id);
    $category->update($request->all());

    return redirect()->route('categories.index')
        ->with('success', 'Category updated successfully');
}

public function destroy($id)
{
    Category::destroy($id);

    return redirect()->back()
        ->with('success', 'Category deleted');
}
}

```

5.1.7 7. ReviewController

```

class ReviewController extends Controller
{
    public function store(Request $request)
    {
        $request->validate([
            'product_id' => 'required|exists:products,id',
            'rating' => 'required|integer|min:1|max:5'
        ]);

        Review::create([
            'user_id' => auth()->id(),
            'product_id' => $request->product_id,
            'rating' => $request->rating,
            'comment' => $request->comment
        ]);

        return redirect()->back()
            ->with('success', 'Review submitted');
    }

    public function destroy($id)
    {
        Review::destroy($id);

        return redirect()->back()
            ->with('success', 'Review deleted');
    }
}

```

5.1.8 8. AddressController

```
class AddressController extends Controller
{
    public function index()
    {
        $addresses = Address::where('user_id', auth()->id())->get();
        return view('addresses.index', compact('addresses'));
    }

    public function create()
    {
        return view('addresses.create');
    }

    public function store(Request $request)
    {
        $request->validate([
            'line1' => 'required|string',
            'city' => 'required|string',
            'country' => 'required|string'
        ]);

        Address::create([
            'user_id' => auth()->id(),
            'line1' => $request->line1,
            'line2' => $request->line2,
            'city' => $request->city,
            'state' => $request->state,
            'zip' => $request->zip,
            'country' => $request->country
        ]);

        return redirect()->route('addresses.index')
            ->with('success', 'Address added');
    }

    public function destroy($id)
    {
        Address::destroy($id);

        return redirect()->back()
            ->with('success', 'Address removed');
    }
}
```

5.1.9 9. VendorController

```
class VendorController extends Controller
```

```

{
    public function index()
    {
        $vendors = Vendor::all();
        return view('vendors.index', compact('vendors'));
    }

    public function create()
    {
        return view('vendors.create');
    }

    public function store(Request $request)
    {
        $request->validate([
            'name' => 'required|string'
        ]);

        Vendor::create($request->all());

        return redirect()->route('vendors.index')
            ->with('success', 'Vendor created');
    }
}

```

5.1.10 10. PaymentController

```

class PaymentController extends Controller
{
    public function store(Request $request)
    {
        $request->validate([
            'order_id' => 'required|exists:orders,id',
            'method' => 'required|string'
        ]);

        Payment::create([
            'order_id' => $request->order_id,
            'method' => $request->method,
            'status' => 'paid',
            'amount' => Order::find($request->order_id)->total_price
        ]);

        return redirect()->route('orders.index')
            ->with('success', 'Payment successful');
    }
}

```

Refined Routing & Controller Hardening

During the alpha-to-stable transition, the backend was hardened by consolidating redundant routes and implementing advanced business logic for inventory management.

5.2.1 Consolidated Routing Architecture

To resolve route shadowing and improve maintainability, the `web.php` structure was re-organized into logical access tiers:

- **Public Tier:** Product discovery and catalog browsing.
- **Auth Tier:** Persistent cart management and checkout flows.
- **Admin Tier:** Secure CRUD operations via `admin` middleware.

5.2.2 Order Lifecycle and Stock Restoration

A robust cancellation mechanism was implemented in the `OrderController`. Using `DB::beginTransaction()`, the system ensures data integrity during cancellations by:

- Validating that only `pending` orders can be cancelled.
- Automatically restoring product `stock` based on `order_items` quantities.
- Updating the order status to `cancelled` in an atomic operation.

5.2.3 Discovery Engine: Search and Filtering

The `ViewController` was enhanced with a dynamic query-building mechanism using Eloquent's `when()` and `filled()` methods. This allows:

- Keyword-based product search by name.
- Category filtering in combination with search queries.
- Efficient query execution without introducing N+1 performance issues.

5.2.4 6. Production MVC Best Practices

- Blade-based rendering (server-side views)
- Redirect + flash messaging for UX feedback
- Authentication handled via Auth facade
- Clean separation between controller and view logic
- Eloquent relationships used for efficient data loading
- Pagination for large datasets
- Form validation inside controller (or Form Requests in future phase)

5.2.5 7. Next Phase Notes

- Build Blade UI templates (layouts, components)
- Implement middleware (auth, admin roles)
- Introduce Form Request validation layer
- Add file upload handling (product images)

Next Phase: System Enhancement and Hardening

5.1.12 Production Notes (ReviewController)

The ReviewController implementation follows standard MVC practices, with additional considerations for production readiness:

- **index()** → Typically restricted to **admin-side usage** for viewing and moderating all reviews.
- **create(\$productId)** → Ensures that every review is explicitly linked to a specific product, preventing orphan records.
- **store()** → Uses **auth()->id()** to securely associate the review with the currently authenticated user, avoiding manual user input and preventing impersonation.
- **update()** → Uses **\$request->only()** instead of **all()** to limit mass assignment and improve data security.
- **destroy()** → Currently performs direct deletion, but in a production environment should be extended with **authorization policies** to ensure users can only delete their own reviews or require admin privileges.

Phase Execution: Frontend and Application Layer

1. Blade UI Templates (View Layer Implementation)

In this phase, the application frontend is developed using Laravel Blade templating. The goal is to create a reusable, maintainable UI structure.

7.1.1 Layout Structure

```
resources/views/  
  layouts/  
    app.blade.php  
  components/  
  products/  
  cart/  
  orders/  
  users/
```

7.1.2 Main Layout Example

```
<!DOCTYPE html>  
<html>  
<head>  
  <title>E-Commerce Platform</title>  
</head>  
<body>  
  
@include('partials.nav')  
  
<div class="container">
```



```

        @yield('content')
    </div>

</body>
</html>

```

2. Middleware Implementation (Authentication and Roles)

Middleware is used to restrict access to certain routes based on authentication and user roles.

7.2.1 Middleware Creation

```
php artisan make:middleware AdminMiddleware
```

7.2.2 Middleware Logic

```

public function handle($request, Closure $next)
{
    if (auth()->user() && auth()->user()->role === 'admin') {
        return $next($request);
    }

    return redirect()->route('home')
        ->with('error', 'Access denied');
}

```

Security Layer: Advanced Form Requests

Validation and authorization concerns were decoupled from controllers using the `StoreProductRequest` class.

- **Granular Authorization:** The `authorize()` method enforces `auth()->user()->role === 'admin'`, preventing unauthorized product creation at the request level.
- **Strict Validation Rules:** Enforced constraints include:
 - `price`: numeric and required
 - `stock`: integer validation
 - `category_id`: existence check in categories table

7.3.1 Controller Usage

```

public function store(StoreProductRequest $request)
{
    Product::create($request->validated());
}

```

4. File Upload Handling (Product Images)

File uploads are handled securely using Laravel storage.

7.4.1 Upload Implementation

```
public function store(Request $request)
{
    $path = $request->file('image')->store('products', 'public');

    Product::create([
        'name' => $request->name,
        'image' => $path
    ]);
}
```

7.4.2 Display in Blade

```

```

5. Production Considerations

- Separation of concerns between controllers, validation, and views
- Secure authentication and role-based access control
- Reusable UI components for scalability
- Proper file storage and access handling

Transition to Full Application Layer

At this stage, the system evolves from a backend-structured application into a complete full-stack solution. Emphasis shifts toward:

- User experience (UI/UX)
- Security and access control
- Codebase modularity and maintainability
- Scalability and production deployment readiness

Blade UI System Implementation

1. View Architecture Overview

The frontend layer is implemented using Laravel Blade templating. The structure is designed for reusability, maintainability, and scalability.

```
resources/views/
  layouts/
    app.blade.php
  partials/
    nav.blade.php
    footer.blade.php
  components/
    product-card.blade.php
  products/
    index.blade.php
```

```
show.blade.php
cart/
orders/
dashboard/
home.blade.php
```

2. Main Layout (Master Template)

All views extend a central layout to ensure consistency.

```
<!DOCTYPE html>
<html>
<head>
    <title>@yield('title', 'E-Commerce')</title>
    <link rel="stylesheet" href="{{ asset('css/app.css') }}">
</head>
<body>

@include('partials.nav')

<div class="container">
    @yield('content')
</div>

@include('partials.footer')

</body>
</html>
```

3. Navigation Bar (Reusable Partial)

```
<nav>
    <a href="{{ route('home') }}">Home</a>
    <a href="{{ route('shop') }}">Shop</a>

    @auth
        <a href="{{ route('cart.index') }}">Cart</a>
        <a href="{{ route('orders.index') }}">Orders</a>
    @endauth

    @guest
        <a href="{{ route('login') }}">Login</a>
    @endguest
</nav>
```

4. Product Listing View (Grid System)

```
@extends('layouts.app')
```

```

@section('content')
<h1>Products</h1>

<div class="grid">
    @foreach($products as $product)
        @include('components.product-card', ['product' => $product])
    @endforeach
</div>

{{ $products->links() }}
@endsection

```

5. Product Card Component (Reusable UI)

```

<div class="product-card">
    

    <h3>{{ $product->name }}</h3>
    <p>{{ $product->price }}</p>

    <a href="{{ route('products.show', $product->id) }}">
        View Details
    </a>
</div>

```

6. Product Detail Page

```

@extends('layouts.app')

@section('content')
<h1>{{ $product->name }}</h1>



<p>{{ $product->description }}</p>
<p>Price: {{ $product->price }}</p>

<form method="POST" action="{{ route('cart.add') }}">
    @csrf
    <input type="hidden" name="product_id" value="{{ $product->id }}">
    <button type="submit">Add to Cart</button>
</form>
@endsection

```

7. Dashboard Structure

```

@extends('layouts.app')

@section('content')

```

```

<h1>Dashboard</h1>

<ul>
    <li><a href="{{ route('products.index') }}">Manage Products</a></li>
    <li><a href="{{ route('orders.index') }}">View Orders</a></li>
    <li><a href="{{ route('users.index') }}">Manage Users</a></li>
</ul>
@endsection

```

8. Blade Concepts Used

- **@extends** → Inherits a layout template
- **@section** / **@yield** → Defines and injects content
- **@include** → Reuses partial views
- **@foreach** → Iterates over collections
- **@auth** / **@guest** → Conditional rendering based on authentication
- **{{ }}** → Outputs escaped data
- **@csrf** → Protects forms from CSRF attacks

9. Production UI Best Practices

- Centralized layout for consistent design
- Reusable components to reduce duplication
- Separation between UI and business logic
- Pagination for performance optimization
- Secure form handling using CSRF protection

Routing Architecture (web.php)

1. Overview

Routing defines how HTTP requests are mapped to controller actions. In this project, routes are structured using Laravel's **web.php** file and follow a clean, modular MVC approach.

Key objectives:

- Maintain readability and organization
- Group related routes logically
- Apply middleware for access control
- Use named routes for maintainability

2. Basic Route Structure

```

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\{
    ViewController,
    ProductController,
    CartController,
    OrderController,

```

```
UserController
};
```

3. Public Routes (Frontend Pages)

```
Route::get('/', [ViewController::class, 'home'])->name('home');
Route::get('/shop', [ViewController::class, 'shop'])->name('shop');
Route::get('/product/{id}', [ViewController::class, 'product'])->name('product.show');

Route::get('/about', [ViewController::class, 'about'])->name('about');
Route::get('/contact', [ViewController::class, 'contact'])->name('contact');
```

4. Authentication-Protected Routes

```
Route::middleware(['auth'])->group(function () {

    Route::get('/cart', [CartController::class, 'index'])->name('cart.index');
    Route::post('/cart/add', [CartController::class, 'add'])->name('cart.add');
    Route::delete('/cart/remove/{id}', [CartController::class, 'remove'])->name('cart.remove');

    Route::get('/orders', [OrderController::class, 'index'])->name('orders.index');
    Route::post('/orders/store', [OrderController::class, 'store'])->name('orders.store');

});
```

5. Admin Routes (Role-Based Access)

```
Route::middleware(['auth', 'admin'])->prefix('admin')->group(function () {

    Route::resource('products', ProductController::class);
    Route::resource('users', UserController::class);

});
```

6. Resource Controllers

Laravel resource routing automatically generates CRUD routes.

```
Route::resource('categories', CategoryController::class);
```

This creates:

- index
- create
- store
- show
- edit
- update
- destroy

7. Named Routes

Named routes improve maintainability and readability.

```
route('products.index')
route('cart.add')
```

Advantages:

- Avoid hardcoded URLs
- Easier refactoring
- Cleaner Blade templates

8. Middleware Usage

Middleware is applied to protect routes and control access.

- **auth** → Ensures user is logged in
- **admin** → Restricts access to administrators

```
Route::middleware(['auth'])->group(function () {
    // Protected routes
});
```

9. Production Routing Best Practices

- Group routes by functionality
- Use prefixes for admin sections
- Apply middleware consistently
- Use resource controllers for CRUD
- Always use named routes

Authorization Policies (Access Control Layer)

1. Overview

Authorization policies are used to control user permissions within the application. They ensure that only authorized users can perform specific actions such as updating or deleting resources.

This introduces a critical **security layer** that operates between controllers and models.

2. Policy Creation

Policies are generated using Artisan:

```
php artisan make:policy ProductPolicy --model=Product
php artisan make:policy ReviewPolicy --model=Review
```

3. ProductPolicy Implementation

```
class ProductPolicy
{
    public function update(User $user, Product $product)
    {
        return $user->role === 'admin';
    }

    public function delete(User $user, Product $product)
    {
        return $user->role === 'admin';
    }
}
```

4. ReviewPolicy Implementation

```
class ReviewPolicy
{
    public function update(User $user, Review $review)
    {
        return $user->id === $review->user_id;
    }

    public function delete(User $user, Review $review)
    {
        return $user->id === $review->user_id
            || $user->role === 'admin';
    }
}
```

5. Policy Registration

Policies must be registered in the AuthServiceProvider:

```
protected $policies = [
    Product::class => ProductPolicy::class,
    Review::class => ReviewPolicy::class,
];
```

6. Using Policies in Controllers

Policies are enforced using the **authorize()** method.

```
public function update(Request $request, $id)
{
    $product = Product::findOrFail($id);

    $this->authorize('update', $product);
}
```



```

        $product->update($request->all());
    }

    public function destroy($id)
    {
        $review = Review::findOrFail($id);

        $this->authorize('delete', $review);

        $review->delete();
    }

```

7. Blade Authorization

Policies can also be applied in Blade templates:

```

@can('update', $product)
    <a href="{{ route('products.edit', $product->id) }}">
        Edit
    </a>
@endcan

```

8. Security Benefits

- Prevent unauthorized data modification
- Enforce ownership rules (user-specific resources)
- Centralize access control logic
- Improve maintainability and scalability

9. Production Best Practices

- Always validate authorization before critical actions
- Avoid placing authorization logic directly in controllers
- Use policies for all user-owned resources
- Combine policies with middleware for layered security

System Architecture Summary

1. Overview

The E-Commerce platform is designed using a layered **MVC (Model-View-Controller)** architecture provided by Laravel. The system separates concerns across multiple layers to ensure scalability, maintainability, and security.

The architecture follows a structured flow:

User Interface (Blade Views) → Controllers → Models (Eloquent ORM) → Database

2. Architectural Layers

11.2.1 2.1 Presentation Layer (View)

Implemented using Laravel Blade templating, this layer is responsible for rendering the user interface.

- Dynamic pages using Blade templates
- Reusable layouts and components
- User interaction through forms and navigation

11.2.2 2.2 Application Layer (Controllers)

Controllers handle incoming HTTP requests and coordinate application logic.

- Process user input
- Interact with models
- Return views or redirects

11.2.3 2.3 Domain Layer (Models)

The domain layer is implemented using Laravel Eloquent ORM.

- Represents database entities
- Defines relationships (One-to-Many, Many-to-Many)
- Encapsulates data access logic

11.2.4 2.4 Data Layer (Database)

The database layer is defined through migrations and schema design.

- Structured relational schema
- Foreign key constraints for integrity
- Optimized for transactional operations (orders, payments)

3. Cross-Cutting Concerns

11.3.1 3.1 Routing

Defines how requests are mapped to controllers using **web.php**.

11.3.2 3.2 Middleware

Handles request filtering and access control.

- Authentication (auth)
- Role-based access (admin)

11.3.3 3.3 Validation Layer

Implemented using Form Request classes to ensure data integrity and clean controller logic.

11.3.4 3.4 Authorization

Policies enforce permissions and ownership rules across the system.

4. Data Flow Execution

A typical request follows this lifecycle:

1. User sends request via browser
2. Route maps request to a controller
3. Middleware validates access permissions
4. Controller processes request
5. Model interacts with database
6. Response returned as a Blade view

5. Security Architecture

Security is enforced at multiple levels:

- Authentication using Laravel Auth system
- Authorization via policies
- CSRF protection for all forms
- Input validation using Form Requests

6. Scalability and Maintainability

The system is designed with future growth in mind:

- Modular controller structure
- Reusable Blade components
- Separation of concerns across layers
- Extendable database schema

7. Final System Perspective

The application has evolved from a basic backend structure into a complete full-stack system:

- Backend: Database, Models, Controllers
- Frontend: Blade UI system
- Security: Middleware and Policies
- Infrastructure: Routing and Validation layers

This layered architecture ensures that the system is production-ready, maintainable, and scalable for future enhancements.

Database Seeding and Test Data Initialization

1. Seeding Strategy (Modular Approach)

To ensure maintainability and scalability, data seeding is implemented using a modular approach. Instead of placing all logic in a single file, multiple seeders are created for each domain.

```
php artisan make:seeder UserSeeder
php artisan make:seeder CategorySeeder
php artisan make:seeder ProductSeeder
php artisan make:seeder ReviewSeeder
php artisan make:seeder OrderSeeder
```

This approach improves organization and allows selective data generation.

2. DatabaseSeeder (Main Entry Point)

The **DatabaseSeeder** class serves as the central execution point for all seeders.

```
public function run()
{
    $this->call([
        UserSeeder::class,
        CategorySeeder::class,
        ProductSeeder::class,
        ReviewSeeder::class,
        OrderSeeder::class,
    ]);
}
```

3. UserSeeder (Role-Based Data)

Users are seeded with different roles to support authentication and authorization testing.

```
use App\Models\User;
use Illuminate\Support\Facades\Hash;

public function run()
{
    User::create([
        'name' => 'Admin User',
        'email' => 'admin@test.com',
        'password' => Hash::make('password'),
        'role' => 'admin'
    ]);

    User::create([
        'name' => 'John Doe',
        'email' => 'user1@test.com',
```

```

        'password' => Hash::make('password'),
        'role' => 'user'
    ]);

    User::create([
        'name' => 'Jane Doe',
        'email' => 'user2@test.com',
        'password' => Hash::make('password'),
        'role' => 'user'
    ]);
}

```

4. CategorySeeder

```

use App\Models\Category;

public function run()
{
    $categories = [
        'Electronics',
        'Clothing',
        'Books',
        'Home & Kitchen'
    ];

    foreach ($categories as $category) {
        Category::create(['name' => $category]);
    }
}

```

5. ProductSeeder

```

use App\Models\Product;

public function run()
{
    Product::create([
        'name' => 'Laptop',
        'price' => 1200,
        'category_id' => 1,
        'stock' => 10,
        'description' => 'High performance laptop'
    ]);

    Product::create([
        'name' => 'T-Shirt',
        'price' => 25,
        'category_id' => 2,
        'stock' => 50
    ]);
}

```

```

]);

Product::create([
    'name' => 'Cooking Book',
    'price' => 15,
    'category_id' => 3,
    'stock' => 30
]);
}

6. ReviewSeeder (Policy Testing Data)

use App\Models\Review;

public function run()
{
    Review::create([
        'user_id' => 2,
        'product_id' => 1,
        'rating' => 5,
        'comment' => 'Excellent product'
    ]);

    Review::create([
        'user_id' => 3,
        'product_id' => 1,
        'rating' => 4,
        'comment' => 'Very good'
    ]);
}

```

This setup ensures:

- Different users own different reviews
- Authorization policies can be properly tested

7. OrderSeeder

```

use App\Models\Order;
use App\Models\OrderItem;

public function run()
{
    $order = Order::create([
        'user_id' => 2,
        'total_price' => 1200,
        'status' => 'completed'
    ]);
}

```

```

        OrderItem::create([
            'order_id' => $order->id,
            'product_id' => 1,
            'quantity' => 1,
            'price' => 1200
        ]);
    }
}

```

8. Running Seeders

```
php artisan migrate:fresh --seed
```

This command:

- Drops all existing tables
- Re-runs all migrations
- Executes all seeders

E-Commerce Platform: Final Configuration and Bug Fixes

1. Problem Identification

Although the core database migrations and models are correctly implemented, several issues prevent the system from functioning as expected:

- Controllers are not retrieving data from the database
- Blade views lack iteration logic for displaying collections
- Missing directory structure for administrative views
- Certain controller methods contain incomplete logic

These issues result in empty views, broken navigation, and improper system behavior.

2. Solutions and Implementation

13.2.1 2.1 Data Retrieval in ViewController

The `shop()` and `product()` methods must retrieve data from the database and pass it to the views.

```
// File: app/Http/Controllers/ViewController.php
```

```

public function shop()
{
    $products = \App\Models\Product::with('category')->get();
    return view('shop', compact('products'));
}

```

```

public function product($id)

```

```
{
    $product = \App\Models\Product::with(['category', 'images'])
        ->findOrFail($id);

    return view('product', compact('product'));
}
```

13.2.2 2.2 Implementing the Shop View

The shop view must iterate over the product collection using Blade syntax.

```
<!-- File: resources/views/shop.blade.php -->

@extends('layouts.app')

@section('content')
    <h1>Shop Catalog</h1>

    <div class="product-grid">
        @foreach($products as $product)
            <div class="product-card">
                <h3>{{ $product->name }}</h3>
                <p>Price: ${{ number_format($product->price, 2) }}</p>

                <a href="{{ route('product.show', $product->id) }}">
                    View Details
                </a>
            </div>
        @endforeach
    </div>
@endsection
```

13.2.3 2.3 Fixing ProductController Logic

The `edit()` method must return a view instead of a redirect to allow user interaction.

```
// File: app/Http/Controllers/ProductController.php

public function edit($id)
{
    $product = Product::findOrFail($id);
    $categories = Category::all();

    return view('products.edit', compact('product', 'categories'));
}
```

13.2.4 2.4 Administrative View Structure

Laravel expects views to follow a structured directory convention. Ensure the following files exist:

- resources/views/products/index.blade.php
- resources/views/products/create.blade.php
- resources/views/products/edit.blade.php

3. Critical Observations

13.3.1 3.1 Middleware Configuration

The admin middleware must be properly registered to avoid access errors.

// File: bootstrap/app.php OR app/Http/Kernel.php

```
'admin' => \App\Http\Middleware\AdminMiddleware::class,
```

13.3.2 3.2 Model Relationship Correction

The Product model relationship for reviews must reflect a one-to-many relationship.

// File: app/Models/Product.php

```
public function reviews()
{
    return $this->hasMany(Review::class);
}
```

4. Outcome

After applying these fixes:

- Products are correctly retrieved and displayed
- Views render dynamic data using Blade
- Administrative routes function properly
- Controller logic aligns with MVC principles

The system now operates as a functional and consistent e-commerce application.

Frontend Enhancement and Feature Expansion Plan

This section defines the progressive enhancement strategy for the E-Commerce platform, moving from basic UI structure to a fully interactive and production-ready interface.

The implementation is divided into three incremental phases to ensure structured development and maintainability.

Option A — User Interface (UI) Improvements

The first phase focuses on improving the visual and structural quality of the application.

- Integration of product images into product cards
- Enhancement of product card styling (spacing, alignment, shadows)
- Implementation of responsive grid layout for mobile, tablet, and desktop views

Goal: Improve user experience and visual presentation without modifying core business logic.

Option B — Functional Feature Enhancements

The second phase introduces core user-facing functionalities that improve usability and navigation.

- Implementation of search bar for product filtering
- Category-based filtering system
- Pagination for product listings to improve performance and usability

Goal: Improve data accessibility and navigation efficiency within the platform.

Option C — Advanced Production-Level Features

The final phase introduces advanced e-commerce functionalities that align the system with production-grade platforms.

- Hero banner carousel for homepage marketing sections
- Discount and promotional pricing system
- Featured product flag stored in database for dynamic homepage control

Goal: Transform the system into a scalable and market-ready e-commerce solution.

Authentication Module Adaptation (Legacy Integration)

The authentication system is adapted from a previous project and refactored to integrate with the current architecture.

14.4.1 Login Interface

```
@extends('layouts.app')

@section('title')
    Login
@endsection

@section('content')

<div class="limiter">
    <form method="POST" action="{{ url('/accessaccount') }}">
        @csrf

        <span>Log in</span>

        <div>
            <input type="text" name="email" placeholder="Email">
        </div>

        <div>
            <input type="password" name="password" placeholder="Password">
        </div>
    </form>
</div>
```

```

        <div>
            <input id="remember" type="checkbox" name="remember">
            <label>Remember me</label>
        </div>

        <div>
            <button type="submit">Login</button>
        </div>

        <div>
            <a href="#">Forgot Password?</a>
        </div>

        <div>
            <a href="{{ url('/signup') }}">Create Account</a>
        </div>
    </form>
</div>

```

```
@endsection
```

14.4.2 Sign Up Interface

```
@extends('layouts.app')
```

```
@section('title')
```

```
    Sign Up
```

```
@endsection
```

```
@section('content')
```

```
@if (Session::has('status'))
```

```
    <div class="alert alert-success">
```

```
        {{ Session::get('status') }}
```

```
    </div>
```

```
@endif
```

```
@if ($errors->any())
```

```
    <div class="alert alert-danger">
```

```
        <ul>
```

```
            @foreach ($errors->all() as $error)
```

```
                <li>{{ $error }}</li>
```

```
            @endforeach
```

```
        </ul>
```

```
    </div>
```

```
@endif
```

```

<div class="limiter">
  <form action="{{ url('/createaccount') }}" method="POST">
    @csrf

    <span>Sign Up</span>

    <div>
      <input type="text" name="name" placeholder="Name">
    </div>

    <div>
      <input type="text" name="email" placeholder="Email">
    </div>

    <div>
      <input type="password" name="password" placeholder="Password">
    </div>

    <div>
      <button type="submit">Sign Up</button>
    </div>
  </form>
</div>

@endsection

```

Implementation Strategy

The development will follow a sequential execution model:

1. UI improvements (visual structure and responsiveness)
2. Functional features (search, filtering, pagination)
3. Advanced production features (carousel, discounts, featured system)
4. Authentication system integration and refinement

Authentication System Completion and Validation

1. Problem Identification

The authentication layer is partially implemented through Blade interfaces (Login and Sign Up views), but lacks the necessary backend logic and routing to function correctly. This results in broken form submissions and incomplete user session handling.

2. Required Fixes and Implementation

15.2.1 2.1 Missing Authentication Controller

There is currently no controller responsible for handling authentication logic.

Required Actions:

- Create an `AuthController`
- Implement the following methods:
 - `register()` – validate input, hash password, create user, log user in
 - `login()` – authenticate using `Auth::attempt()`
 - `logout()` – terminate user session

Impact: Without this controller, authentication forms cannot be processed.

15.2.2 2.2 Password Hashing Verification

User passwords must be securely hashed before being stored in the database.

Verification Required:

- Ensure password hashing is implemented using `Hash::make()`
- Alternatively, define proper casting or mutators in the `User` model

Impact: Storing plain-text passwords introduces critical security vulnerabilities.

15.2.3 2.3 Missing POST Routes

The application currently defines only GET routes for authentication views.

Required Routes:

```
Route::post('/createaccount', [AuthController::class, 'register']);
Route::post('/accessaccount', [AuthController::class, 'login']);
Route::post('/logout', [AuthController::class, 'logout'])->name('logout');
```

Impact: Form submissions will fail without corresponding POST route handlers.

15.2.4 2.4 Role Management Strategy

The system includes role-based access control through `AdminMiddleware`, but role assignment during registration is not clearly defined.

Verification Required:

- Ensure the `users` table contains a `role` column
- Confirm a default value (e.g., `'user'`) is assigned

Impact: Incorrect role assignment may break authorization logic.

15.2.5 2.5 Layout Adaptation for Authentication State

The main layout must dynamically adapt based on authentication status.

Required Behavior:

- Show `Login` and `Register` links for guests
- Show `Logout`, `Cart`, and user-specific links for authenticated users

Impact: Improves user experience and ensures proper navigation flow.

15.2.6 2.6 Session and Validation Feedback

The frontend already supports error and session messages, but backend validation must be properly implemented.

Required Actions:

- Use `$request->validate()` in controller methods
- Return validation errors to Blade views

Impact: Ensures users receive meaningful feedback on failed actions (e.g., invalid input, duplicate email).

3. Outcome

After implementing these fixes:

- Authentication system becomes fully functional
- User registration and login are securely handled
- Role-based access control is enforced
- UI dynamically adapts to user state
- Error handling improves usability and reliability

The system now supports a complete and secure authentication workflow aligned with production standards.

Implementation Progress: Authentication and User Lifecycle Management

Completed Milestones

The authentication system and user management infrastructure are now fully operational. The implementation follows standard security protocols for modern Laravel applications, including secure credential handling, session management, and role-based access control.

16.1.1 1. Authentication Controller (AuthController)

A centralized controller manages the full authentication lifecycle:

- **Registration:** Validates user input, creates user records with a default `'user'` role, and automatically logs the user into the system.
- **Login:** Authenticates users using `Auth::attempt()` and regenerates sessions to prevent session fixation attacks.
- **Logout:** Safely terminates user sessions, invalidates session data, and regenerates CSRF tokens.
- **Remember Me Support:** Persistent login functionality implemented using session-based token storage.

16.1.2 2. Secure Data Handling and Model Configuration

The `User` model and database schema have been aligned with Laravel security standards:

- **Mass Assignment Protection:** The `role` attribute has been added to the `fillable` array to allow controlled assignment during user creation.
- **Password Security:** Password hashing is handled automatically via the `hashed` cast, eliminating the risk of plain-text storage or double hashing.
- **Hidden Attributes:** Sensitive fields such as passwords are excluded from serialized outputs.
- **Schema Consistency:** The `users` table includes a `role` column with a default value of `'user'`, supporting role-based authorization.

16.1.3 3. Routing and Middleware Architecture

The routing layer has been structured into clear access tiers:

- **Guest Routes:** Login and signup pages are accessible without authentication and use named routes for redirection.
- **Authenticated Routes:** Cart and order operations are protected using the `auth` middleware.
- **Administrative Routes:** Product and user management routes are grouped under an `admin` prefix and protected by custom middleware.

Middleware Implementation:

- A custom `AdminMiddleware` validates user roles.
- Middleware is registered in `bootstrap/app.php` using alias mapping.

16.1.4 4. Dynamic Layout and Session Awareness

The main layout system has been enhanced to reflect authentication state:

- Conditional rendering using Blade directives (`@auth`, `@guest`)
- Dynamic navigation displaying login/register or user/logout options
- Integration of session feedback messages for user actions

16.1.5 5. Backend Integration for Product Display

The `ViewController` has been refactored to support dynamic data rendering:

- Product listings are retrieved using Eloquent ORM
- Relationships such as `category`, `images`, and `reviews` are eagerly loaded
- Product detail pages now display structured and relational data instead of static placeholders

16.1.6 6. Database Seeding and Testing Infrastructure

A modular seeding strategy has been implemented to support development and testing:

- Separate seeders for users, categories, products, reviews, and orders
- Predefined roles (admin/user) for testing authorization logic
- Relational data ensures realistic testing scenarios (e.g., user-owned reviews)
- Database reset and seeding handled via `php artisan migrate:fresh --seed`

16.1.7 7. Error Handling and Validation Layer

Robust validation and feedback mechanisms have been integrated:

- Input validation handled using `$request->validate()`
- Validation errors propagated to Blade views
- Session-based success and error messaging implemented

System Status

At this stage, the application has transitioned from a structural backend implementation to a fully functional system with:

- Secure authentication and session management
- Role-based access control (RBAC)
- Dynamic product data rendering
- Modular database seeding for testing
- Structured routing and middleware enforcement

Remaining Implementation Scope

The following components remain to achieve full production readiness:

- Cart and checkout workflow completion
- Order lifecycle management (creation, status tracking)
- Review system enforcement using policies
- UI enhancement (responsive design and layout improvements)
- Advanced features (search, filtering, pagination)

Cart and Checkout System Implementation

1. Overview

The cart and checkout system enables users to manage selected products and convert them into orders. This module integrates tightly with authentication, product inventory, and order management.

2. Cart Management Logic

The cart system is designed to support persistent user-specific storage and dynamic item management.

17.2.1 Core Behaviors

- Add products to cart
- Automatically increase quantity if product already exists in cart
- Remove items from cart
- Display total cart value

17.2.2 Cart Initialization (Critical)

To prevent null reference errors, a cart instance is created dynamically if it does not exist:

```
$cart = auth()->user()->cart ?? Cart::create([
    'user_id' => auth()->id()
]);
```

Purpose: Ensures every authenticated user always has an associated cart.

3. Checkout and Order Creation

The checkout process converts cart data into a structured order.

17.3.1 Checkout Flow

1. User initiates checkout
2. System validates cart contents
3. Order record is created
4. Cart items are transferred to order items
5. Cart is cleared after successful transaction

17.3.2 Order Creation Logic

```
$order = Order::create([
    'user_id' => auth()->id(),
    'total_price' => $total,
    'status' => 'pending'
]);
```

Purpose: Establishes a persistent order record tied to the authenticated user.

4. Data Integrity and Validation

To ensure system reliability, several validation mechanisms are implemented.

17.4.1 Stock Protection

```
if ($product->stock < $request->quantity) {
    return back()->withErrors('Not enough stock');
}
```

Purpose: Prevents overselling of products beyond available inventory.

17.4.2 Empty Cart Prevention

```
if ($cart->items->isEmpty()) {
    return back()->withErrors('Cart is empty');
}
```

Purpose: Ensures that orders cannot be created without valid items.

5. System Integration

The cart and checkout module integrates with:

- **Authentication:** Only logged-in users can access cart and checkout features
- **Products:** Cart items reference valid product records
- **Orders:** Cart transitions into order lifecycle management

6. Outcome

At this stage, the system supports:

- Persistent user-specific carts
- Dynamic product quantity handling
- Secure checkout process
- Order creation with relational integrity

The platform now includes a complete transactional flow from product selection to order generation.

Shopping Cart and Order Processing System

Overview

At this stage, the application transitions from static product display to a fully interactive e-commerce workflow. Users can now add products to a cart, manage quantities, and complete a checkout process that generates persistent orders.

Cart Management Implementation

The cart system is designed to maintain a one-to-one relationship between a user and their active cart session.

18.2.1 Core Features

- Add products to cart
- Prevent duplicate entries by increasing quantity
- Remove items from cart
- Display cart contents dynamically

18.2.2 Cart Initialization Logic

To prevent runtime errors caused by missing carts, a fallback mechanism ensures a cart is always available:

```
$cart = auth()->user()->cart ?? Cart::create([
    'user_id' => auth()->id()
]);
```

This guarantees that cart operations never fail due to null references.

18.2.3 Stock Protection

Before adding items to the cart, the system validates available inventory:

```
if ($product->stock < $request->quantity) {  
    return back()->withErrors('Not enough stock');  
}
```

This prevents users from reserving unavailable quantities.

Order Processing (Checkout Flow)

The checkout process converts cart data into a structured order while maintaining data integrity.

18.3.1 Checkout Workflow

- User initiates checkout
- Total price is calculated
- Order record is created
- Cart items are transferred to order items
- Product stock is updated
- Cart is cleared

18.3.2 Order Creation

```
$order = Order::create([  
    'user_id' => auth()->id(),  
    'total_price' => $total,  
    'status' => 'pending'  
]);
```

18.3.3 Data Migration (Cart → Order)

Each cart item is transformed into a persistent order item:

```
OrderItem::create([  
    'order_id' => $order->id,  
    'product_id' => $item->product_id,  
    'quantity' => $item->quantity,  
    'price' => $item->product->price  
]);
```

18.3.4 Stock Update

```
$item->product->decrement('stock', $item->quantity);
```

18.3.5 Cart Cleanup

```
$cart->items()->delete();
```

Data Integrity and Transaction Safety

To ensure consistency, the checkout process is wrapped in a database transaction:

```
DB::transaction(function () {  
    // Order logic  
});
```

This guarantees:

- No partial orders are created
- Stock levels remain accurate
- Failures rollback automatically

User Experience Considerations

- Users receive feedback messages for all actions (add/remove/checkout)
- Empty cart protection prevents invalid orders
- Product availability is enforced at both cart and checkout levels

System Readiness

At this stage, the application supports:

- Complete shopping workflow
- Persistent order storage
- Inventory-aware transactions
- Secure user-bound operations

This marks the transition from a structural prototype to a functionally complete e-commerce backend.

Controller Methods and Laravel Terminology

This section explains key Laravel functions and patterns used throughout controller implementation.

Core Helper Functions

redirect() Used to navigate the user to another route or page after an action. Commonly used after form submission.

```
return redirect()->route('products.index');
```

view() Returns a Blade template (UI page) with optional data.

```
return view('products.index', compact('products'));
```

compact() A PHP helper that converts variables into an associative array.

```
compact('products')
```

Eloquent ORM Methods

create() Inserts a new record into the database using mass assignment.

```
Product::create($data);
```

update() Updates an existing database record.

```
$product->update($request->all());
```

destroy() Deletes a record by ID.

```
Product::destroy($id);
```

findOrFail() Retrieves a record or throws a 404 error if not found.

```
Product::findOrFail($id);
```

with() Eager loads relationships to improve performance.

```
Product::with('category')->get();
```

Request Handling

Request Handles HTTP input data from forms.

validate() Validates incoming request data before processing.

```
$request->validate([
    'name' => 'required|string'
]);
```

Authentication

auth()->id() Returns the currently authenticated user's ID.

```
$userId = auth()->id();
```

Session Flash Messages

with() Stores temporary session data (e.g., success messages).

```
->with('success', 'Operation completed');
```

Routing Integration

route() Generates URLs based on named routes.

```
route('products.index');
```

Professional UI/UX Implementation Summary

The frontend was transformed from a basic prototype into a high-end editorial interface. This phase emphasized visual hierarchy, fault tolerance, and production-grade UI patterns.

Core Design System

- **Typography:** Adoption of *Plus Jakarta Sans* to achieve a modern premium aesthetic.
- **Glassmorphism:** Implementation of backdrop blur effects (`blur(12px)`) and translucent surfaces across navigation and filtering components.
- **Layout Stability:** Use of aspect-ratio locked containers to prevent layout shifts during image loading.

Robust Image & Asset Handling

To resolve image visibility issues and ensure graceful degradation:

- **Path Standardization:** All assets use `asset('storage/...')` aligned with Laravel's `php artisan storage:link` configuration.
- **Fault Tolerance:** All image components implement an `onerror` handler to replace missing assets with high-quality placeholders.

Enhanced Component Lifecycle

- **Multi-Angle Product Gallery:** Interactive main image stage with thumbnail navigation for multiple product views.
- **Persistent Cart UI:** Split-grid cart layout with sticky order summary to improve checkout usability.
- **Order History Visualization:** High-contrast status indicators (e.g., `status-pending`, `status-completed`) for rapid interpretation.

Implementation Note: The backend logic (transaction handling, stock restoration, and secure validation layers) demonstrates mastery of Laravel's architectural principles. The frontend implementation follows modern UI/UX standards to deliver a production-ready, conversion-optimized user experience.

Luxury-Tier UI Finalization

To elevate the platform from a functional prototype to a high-end commercial interface, a final sweep was conducted to apply *Tier 3* design principles across all secondary and administrative views.

20.4.1 Public Storytelling and Support

The **about** and **contact** pages were transformed into brand-building assets:

- **About View:** Implemented a cinematic editorial layout, featuring a manifesto section and a grid-based presentation of brand values.
- **Contact View:** Created a dual-pane layout separating physical concierge information from a high-contrast client inquiry form.

20.4.2 Executive Administrative Dashboard

The administrative layer was overhauled to provide an "Executive Command Center" experience:

- **Command Dashboard:** Replaced basic links with elevated action cards, each containing descriptive metadata and hover-state transitions.
- **Inventory Ledger:** Redesigned the product index as a professional management table, including semantic **status-pills** (e.g., *Low Stock*, *Out of Stock*) and interactive image previews.
- **Asset Curation:** The **create** and **edit** forms were re-engineered into a centered "Editor's Stage," featuring drag-and-drop imagery zones and structured data fields.

20.4.3 UI Stability and Grid Logic

Critical refinements were applied to the Discovery section on the home page:

- **Grid Overlap Resolution:** Fixed layout collisions in the "Latest Drops" section by enforcing strict CSS grid constraints and overflow management.
- **Semantic Phrasing:** Updated filtering interactions to use the customer-requested "Apply Filters" nomenclature, improving UX clarity.

Conclusion

This document represents the ****execution layer**** of the system design. Each section will be progressively expanded as implementation continues, moving from architecture → database → API → full system deployment.